Relatório

Laboratórios de Informática III

Grupo 9:
Rafael Figueiras Esteves
João Pedro Cunha Cerqueira
Nuno Henrique Dias Pereira



Universidade do Minho
Escola de Engenharia

Índice

Introdução..... 3
Sistema..... 3
    Modularidade.....8
    Encapsulamento.....8
    Metodologia de Testes e Análise de Desempenho..... 8
Conclusão..... 9

Introdução

Este relatório apresenta o trabalho desenvolvido no âmbito da segunda fase do projeto de Laboratórios de Informática, cujo objetivo central é a consolidação de competências em programação. O projeto consiste na implementação de um sistema robusto de gestão e análise de dados de tráfego aéreo, capaz de processar informações sobre aeroportos, voos, passageiros e reservas.

Enquanto a primeira etapa do trabalho focou-se no estabelecimento das fundações do sistema, nomeadamente na criação de uma arquitetura modular, na separação entre interface e implementação e na validação rigorosa dos datasets de entrada, esta segunda fase trouxe desafios de complexidade significativamente superior. O aumento substancial do volume de dados a processar exigiu uma mudança de paradigma: o foco deixou de ser apenas a correção funcional para passar a ser, principalmente, a eficiência e o desempenho.

Para responder a este desafio, tivemos de reformular e otimizar as estruturas de dados várias vezes para testar quais eram as mais eficientes para cada caso, substituindo soluções lineares simples por estruturas mais avançadas, como hash tables ou árvores. Adicionalmente, o sistema foi expandido com novas funcionalidades, incluindo a implementação de três novas queries mais complexas, a garantia de documentação formal do código e a criação de um "modo interativo", que permite ao utilizador navegar através do terminal.

Ao longo deste documento, detalharemos a arquitetura final do projeto, justificando as escolhas técnicas tomadas para garantir o encapsulamento e a modularidade exigidos. Apresentaremos também uma análise crítica do desempenho alcançado, suportada por testes, demonstrando como as otimizações implementadas permitiram tratar grandes volumes de dados sem comprometer a estabilidade do programa ou gerar memory leaks.

Sistema

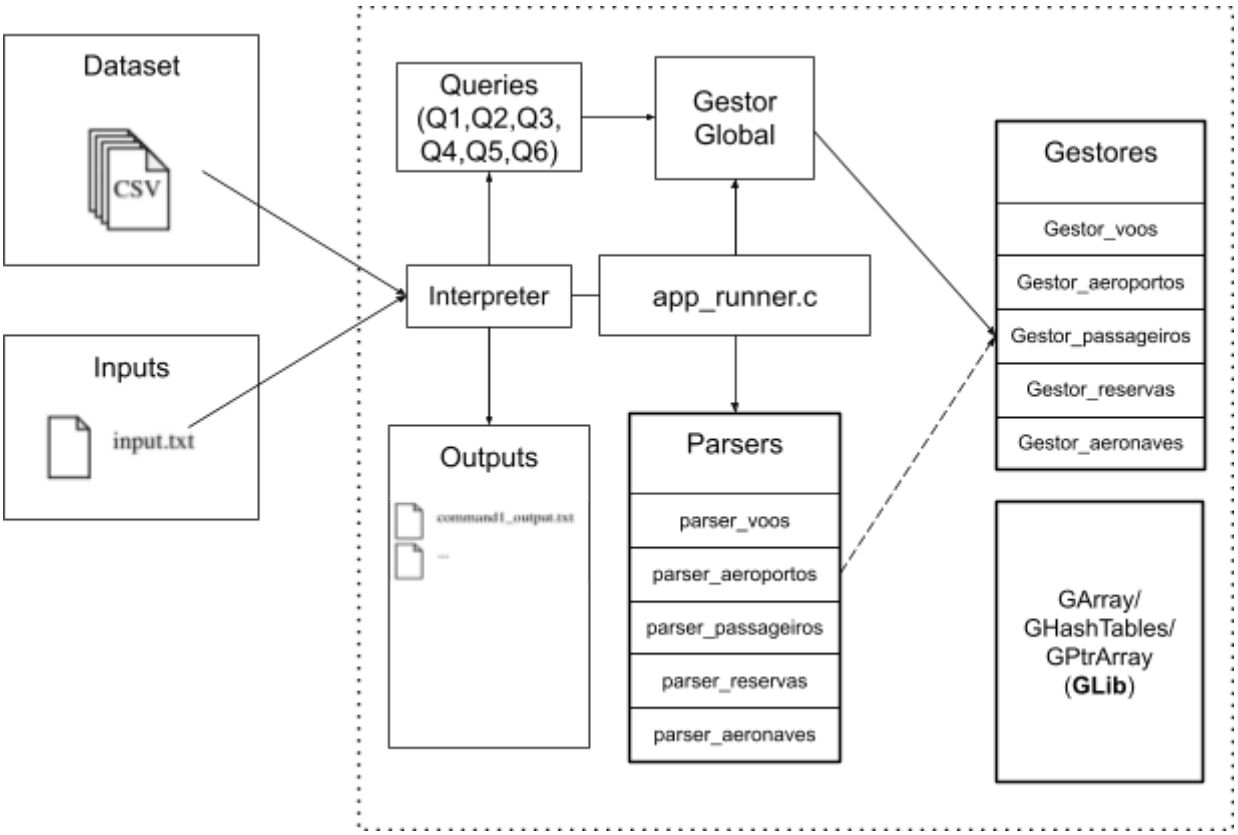
Nesta fase do trabalho decidimos focar essencialmente na otimização do tempo de execução das queries e no uso de memória.

O nosso programa começa por executar a função main. A partir daí, chama o módulo “app_runner”, que é responsável por carregar o dataset: abre os ficheiros CSV e, para cada tipo de ficheiro, passa o controlo ao respetivo parser.

Os parsers lêem o CSV linha a linha e fazem a separação dos campos. Para cada linha, validam os dados (e, quando necessário, fazem validações cruzadas usando informação já carregada). Se a linha for válida, criam a entidade correspondente e enviam-na para o gestor dessa entidade, se for inválida, a linha é escrita no ficheiro de erros correspondente.

Cada gestor (aeroportos, voos, aeronaves, passageiros e reservas) recebe as entidades válidas e guarda-as nas suas estruturas de dados internas hashtables e em alguns casos, índices auxiliares para acelerar consultas. No topo existe o gestor global, que agrega todos os subgestores e fornece acesso controlado a cada um deles.

Depois de o carregamento terminar, o interpretador lê o ficheiro de inputs (comandos) e, para cada linha, identifica qual a querie a executar. Em seguida, chama a função da querie correspondente, passando o gestor global. A querie usa o gestor global para obter acesso aos subgestores e aos dados/índices necessários, calcula o resultado pedido e escreve o output no ficheiro resultados/commandN_output.txt (um ficheiro por comando).



A **Querie 1** interpreta a linha do comando, identifica o código IATA do aeroporto (normalizado para maiúsculas) e define o separador a usar no output (; por defeito, ou = no modo alternativo com S).

Em seguida, a querie constrói o caminho do ficheiro de saída com base em Nout e abre-o em modo de escrita. Procura o aeroporto no gestor através do IATA e, caso este não seja encontrado, escreve uma linha vazia. Se existir, os contadores de “aterragens” e “partidas” são obtidos através da cache da Querie 1. A função q1_lookup_counts garante que os contadores são obtidos de uma GHashTable (q1_cache->iata_to_counts) que associa cada IATA a uma estrutura Q1Counts { aterragem, partida }. Essa hash table é construída apenas uma vez na primeira vez que for necessária, através de q1_build_cache_if_needed.

A construção da cache percorre todas as reservas (ReservaPtrManager), porque é nas reservas que existe a lista de voos associada (flight_ids). Para cada reserva, lê a string flight_ids e extrai até dois identificadores de voo com extract_flight_ids_fast, que procura rapidamente valores entre aspas (simples ou duplas) e guarda IDs de 7 ou 8 caracteres. Para cada flight_id extraído, a querie procura o voo no gestor de voos (voo_ptr_manager_find_by_id). Se o voo não existir ou

estiver com estado CANCELLED, é ignorado. Caso contrário, lê o aeroporto de origem e de destino do voo e incrementa os contadores correspondentes na hashtable: incrementa a partida para o IATA de origem e aterragem para o IATA de destino. Quando termina a varredura das reservas, marca a cache como construída, e nas execuções seguintes a Querie 1 já não volta a percorrer reservas/voo, faz apenas um lookup direto na hashtable para obter os contadores.

Por fim, com o aeroporto validado e os contadores obtidos, a querie gera a parte textual do aeroporto através de airport_summary_delim (que escreve num buffer os campos do aeroporto usando o separador escolhido) e escreve no ficheiro a linha final no formato resumo_do_aeroporto<sep>aterragens<sep>partidas, terminada por \n. Fecha o ficheiro e incrementa *Nout para garantir que o próximo comando gere command(Nout+1)_output.txt.

Usamos **Cache estática** pois a Querie 1 pode ser chamada muitas vezes, compensa manter o resultado agregado em memória para reutilização.

A **Querie 2** processa a linha de input (comando 2), extrai o número N de resultados e, opcionalmente, um filtro de fabricante. O modo 'S' alterna o separador de output para '='.

A função obtém os gestores de voos e aeronaves e define o caminho do ficheiro de saída (resultados/command%d_output.txt). Se os gestores não existirem ou não houver aeronaves, gera um ficheiro vazio e incrementa *Nout.

Se os dados existirem, constrói um array dinâmico (GPtrArray com g_free automático) para armazenar estruturas AircraftCount (ponteiro para a Aeronave e contagem de voos). Percorre todas as aeronaves, aplica o filtro de fabricante (se existir) e usa voo_ptr_manager_get_flight_count_by_aircraft para obter o número de voos (não cancelados). Aeronaves com count <= 0 são ignoradas, as restantes são adicionadas ao array.

O array é então ordenado com g_ptr_array_sort e a função de comparação compare_aircraft_counts: primeiro por número de voos (decrescente) e, em caso de empate, por id da aeronave (crescente).

Finalmente, escrever_resultados abre o ficheiro, escreve até N linhas no formato id<sep>fabricante<sep>modelo<sep>count, ou uma linha vazia se N ou o array estiverem vazios. O array é libertado, e se a escrita for bem-sucedida, *Nout é incrementado.

A **Querie 3** lê a linha do comando e extrai as duas datas do intervalo (begin_date e end_date), validando o formato YYYY-MM-DD e garantindo que a data inicial não é posterior à final. Se houver qualquer erro de parsing ou o comando estiver incompleto, o programa cria o ficheiro resultados/commandX_output.txt e escreve apenas uma linha vazia. Tal como nas outras queries, também suporta o modo alternativo S, que altera o separador no output de ; para =.

Com o intervalo válido, a querie prepara uma hash table contagens que associa cada IATA de aeroporto de origem a uma estrutura AirportCount com um contador de partidas. Para contar apenas os voos relevantes sem percorrer todo o dataset, a implementação usa o índice voos_por_data do gestor de voos, que é uma GTree ordenada por data, a função g_tree_foreach percorre as entradas da árvore e, para cada dia, a função processar_voos_range aplica duas otimizações: se a data do nó ainda for anterior ao início do intervalo, continua sem contar; se a data já for posterior ao fim do intervalo, termina imediatamente a iteração. Quando a data está dentro do intervalo, a callback percorre a lista de voos desse dia (GPtrArray), obtém o IATA de origem de cada

voo e incrementa o contador correspondente na hash table (criando uma nova entrada caso ainda não exista).

Depois de contabilizar todas as partidas no intervalo, a querie percorre a hash table para encontrar o aeroporto com o maior número de partidas, aplicando o desempate pelo menor código IATA lexicograficamente quando as contagens são iguais. Em seguida abre o ficheiro de output e escreve o resultado: obtém os dados de texto do aeroporto através de airport_summary_delim, remove o último campo do resumo (porque a Querie 3 não inclui o "tipo" do aeroporto) e imprime resumo_sem_tipo<sep>count. Se não existir nenhum resultado válido, escreve apenas uma linha vazia. Por fim, fecha o ficheiro, destrói as estruturas auxiliares (contagens e datas) e incrementa *Nout para manter a numeração dos outputs.

Nesta **Querie 3** foi-nos aconselhado pelos professores na 1 defesa do projeto a usar GTree para consultar datas.

A **Querie 4** interpreta a linha do comando, podendo receber opcionalmente um intervalo de datas [begin_date end_date], e também suporta o modo alternativo S para mudar o separador do output (por defeito ;, ou = quando é usado S). Em qualquer caso, prepara o caminho do ficheiro resultados/commandX_output.txt com base em *Nout, se o comando tiver datas mal formatadas, se begin_date > end_date, ou se for impossível trabalhar com os dados, a querie escreve apenas uma linha vazia nesse ficheiro e termina, garantindo sempre output.

O processamento principal assenta numa cache (Q4Cache) construída apenas uma vez. Na primeira execução, build_cache_if_needed percorre todas as reservas e, para cada uma, valida o passageiro através do document_number, extrai o primeiro flight_id associado à reserva, e obtém o voo correspondente no gestor de voos. A semana é determinada pela data de partida estimada do voo (departure), convertida para "número de dias" e arredondada para o domingo dessa semana (a semana começa no domingo e termina no sábado). Em seguida, para essa semana, acumula-se o gasto do passageiro somando reserva_get_price(r) num mapa por semana, ou seja, constrói-se uma estrutura do tipo week_start -> (passageiro -> total_gasto).

Depois de acumulados os totais semanais, a cache transforma esses totais no "Top 10" de cada semana: para cada semana, percorre os passageiros dessa semana e mantém um top 10 ordenado por maior gasto total e, em caso de empate, pelo menor document_number. Esse top 10 é guardado em week_to_top10 (uma hash table que mapeia a semana para um GPtrArray com os passageiros do top 10, já ordenados) e as semanas existentes são guardadas num array ordenado sorted_week_starts, o que permite iterar semanas de forma eficiente.

Na execução efetiva da querie, depois de garantir que a cache está pronta, define-se o conjunto de semanas a considerar: se o utilizador forneceu um intervalo, a querie calcula as semanas abrangidas (incluindo semanas parcialmente cobertas no início ou no fim). Se não forneceu, considera desde a primeira até à última semana presentes na cache. A querie percorre semana a semana (saltando de 7 em 7 dias), obtém o top 10 dessa semana e incrementa um contador global counts para cada passageiro que aparece nesse top 10, ou seja, conta "quantas semanas esteve no top 10". No final escolhe o passageiro com maior count_top_10 e, em caso de empate, seleciona o que tiver o menor document_number. Por fim escreve uma única linha no ficheiro de output com document_number;first_name;last_name;dob;nationality;count_top_10 (usando o separador escolhido), fecha o ficheiro, liberta estruturas auxiliares e incrementa *Nout.

Cache estática (Q4Cache): esta querie usamos cache pois ela agrega os dados por semana e por passageiro. Como pode ser chamada várias vezes (com/sem intervalo), compensa pré-computar e reutilizar.

A **Querie 5** processa a linha do comando 5, extrai o valor N (número de companhias a listar) e, tal como nas restantes queries, suporta o modo S para mudar o separador do output de ; para =. Se o argumento estiver em falta, não for numérico ou for <= 0, a execução termina produzindo apenas uma linha vazia no ficheiro resultados/commandX_output.txt. Com um N válido, a querie obtém o gestor de voos e constrói (apenas uma vez) uma vista ordenada em cache. Essa lista é construída a partir da hash table delay_stats_by_airline do gestor de voos, que associa cada companhia a estatísticas agregadas (delayed_flights_count e total_delay_minutes) acumuladas previamente. Para cada companhia com pelo menos um voo atrasado, calcula-se o atraso médio como total_delay_minutes / delayed_flights_count e aplica-se um arredondamento a 3 casas decimais através da função round3. Cada entrada é guardada num AirlineDelayView e adicionada a um GPtrArray, que é depois ordenado por atraso médio decrescente e, em caso de empate, por ordem alfabética da companhia. Na fase final, a querie abre o ficheiro de output e escreve até N linhas no formato airline<sep>delayed_flights_count<sep>average_delay com três casas decimais, se a cache estiver vazia, escreve apenas \n. O ficheiro é fechado e *Nout é incrementado para avançar para o próximo commandX_output.txt.

Agregado no gestor (delay_stats_by_airline em GHashTable): os somatórios necessários (nº de voos atrasados e total de minutos) são atualizados no carregamento. Assim, a querie não precisa percorrer voos para cada execução.

A **Querie 6** interpreta a linha do comando 6, lê a nacionalidade (o resto da linha, removendo espaços no fim) e também suporta o modo S para escolher o separador (; ou =). Se a nacionalidade estiver vazia, a querie gera o ficheiro resultados/commandX_output.txt e escreve apenas uma linha vazia. Caso exista uma nacionalidade válida, a execução consulta uma cache (Q6Cache) que é construída apenas uma vez: a cache contém uma hash table nat_to_airport_counts que mapeia cada nacionalidade para outra hash table com contagens de aterragens por aeroporto de destino (airport_iata -> count). Na construção da cache, a querie percorre todas as reservas, obtém o passageiro pelo document_number no gestor de passageiros, lê a nacionalidade desse passageiro e extrai até dois flight_id da reserva (função extract_flight_ids). Para cada voo associado, se existir e não estiver cancelado, obtém-se o IATA do destino e incrementa-se o contador para o par (nacionalidade, destino), criando as estruturas intermédias quando necessário. Depois de a cache estar pronta, a querie faz lookup da nacionalidade pedida: se não existir mapa ou estiver vazio, escreve linha vazia. Caso exista, percorre as entradas desse mapa para encontrar o aeroporto com maior número de aterragens e, em empate, escolhe o menor IATA por ordem lexicográfica (strcmp). Por fim grava no output a linha code<sep>arrival_count terminada com \n, fecha o ficheiro e incrementa *Nout.

Modularidade

A solução foi organizada por responsabilidades bem definidas, separando o código em módulos independentes e fácil interpretação. As estruturas do domínio (aeroportos, aeronaves, voos, passageiros e reservas) estão isoladas em “entidades” (entidades), enquanto a lógica de armazenamento/consulta dessas entidades está nos “gestores” (gestores). O carregamento de dados é tratado à parte pelos “parsers” (parsers), e a execução das queries fica concentrada nos módulos de “queries” (queries). Por fim, o módulo de orquestração (app_runner.c) coordena o carregamento e a execução dos comandos sem conhecer detalhes internos das estruturas.

Esta separação reduz o acoplamento: por exemplo, as queries não precisam de saber como os dados são lidos do CSV, e os parsers não precisam de conhecer como as queries funcionam. Cada camada depende apenas de interfaces públicas expostas em headers na pasta include e não de implementações concretas.

Encapsulamento

O encapsulamento foi garantido ao esconder a representação interna das estruturas de dados. As entidades e gestores expõem operações “seguras” via funções públicas (criação, destruição e getters), mantendo os campos internos privados dentro dos ficheiros .c. Isto evita que outros módulos acedam diretamente a membros das struct, o que reduz a quantidade de erros.

Um exemplo claro é o parsing: a leitura/normalização de linhas CSV e o “split” em campos foi abstraído num parser genérico (csv_parser.c), que recebe callbacks para validar, criar e inserir entidades. Assim, cada parser específico implementa apenas a lógica do seu tipo (ex.: “validar campos” e “construir a entidade”), enquanto o fluxo comum de leitura/erros/rejeições fica encapsulado e reutilizável. Do lado dos gestores, o resto do programa interage por funções de inserção e procura, sem depender de como a coleção é guardada internamente.

Metodologia de Testes e Análise de Desempenho

Características dos computadores dos integrantes do grupo

Aluno	**Processador**	**Memória Ram**
Rafael Esteves	AMD Ryzen™ 7 7840HS	32GB
João Cerqueira	AMD Ryzen™ 5 7000HS	16GB
Henrique Pereira	Intel Core Ultra 7 155H	32GB

Tempos de execução das Queries (Média de 3 execuções)

Tempo	Rafael Esteves	João Cerqueira	Henrique Pereira
Q1	0.061 s	0.052 s	0.042 s
Q2	0.00064 s	0.001 s	0.00087 s
Q3	0.077 s	0.100 s	0.062 s
Q4	0.075 s	0.083 s	0.066 s
Q5	0.00002 s	0.000033 s	0.000015 s
Q6	0.110 s	0.096 s	0.088 s
Total:	16.129 s	16.649 s	12.918 s

Memória utilizada (Média de 3 execuções)

Aluno	**Uso de Memória**
Rafael Esteves	3081.61 MB
João Cerqueira	3081.08 MB
Henrique Pereira	3080.63 MB

Conclusão

A segunda fase do trabalho deu mais importância à organização em grandes volumes de dados. Enquanto a primeira fase provou a funcionalidade, esta demonstrou a capacidade de otimização e eficiência do sistema em contextos de maior exigência.

O desafio central foi a otimização de desempenho. A adoção de estruturas de dados avançadas (como hash tables ou árvores) foi crucial para otimizar os requisitos de tempo e memória. Confirmou-se que a escolha de estruturas de dados é o fator que distingue um programa funcional de um eficiente.

Outro ponto de destaque foi a melhoria na praticidade, nomeadamente pela implementação do modo interativo para navegação de dados via terminal. O reforço do encapsulamento e a documentação Doxygen garantiram a organização, legibilidade e modularidade do código.

Em resumo, o projeto simulou um ambiente de desenvolvimento real, focando na adaptação a novos requisitos e na gestão de grandes dados. O resultado é um sistema que cumpre todos os objetivos, sendo uma solução modular, documentada e otimizada para a gestão de dados de tráfego aéreo.